

Skills — Механизм скиллов OpenCode в проекте AIS

> **Версия:** 1.0 от 15.09.2026

> **Родительский том:** [Главный том документации](#)

1. Что такое скилл (skill) в OpenCode

Скилл — это набор инструкций для LLM-агента, который загружается **по требованию**, а не всегда. Каждый скилл — это папка с файлом `SKILL.md`, содержащим фронтmatter (метаданные) и тело инструкций в формате Markdown.

1.1. Физическая структура

```
.opencode/  
  
└─ skills/  
    └─ my-skill/  
        └─ SKILL.md      ← единственный обязательный файл  
  
    └─ another-skill/
```

1.2. Формат SKILL.md

```

---

name: my-skill

description: Описание что делает скилл и когда его загружать.

---

# Название скилла

Тело инструкций в Markdown: примеры, правила, ссылки.
```

Обязательные поля фронтматтера:

Поле	Описание	Пример
name	Имя скилла (строчные, через дефис, ≤64 символа)	pb-object-skill
description	Описание: что делает И КОГДА загружать. Ключевые слова в начале	Используй ТОЛЬКО при работе с объектами PowerBuilder...

1.3. Как OpenCode находит скиллы

Конфигурация в `opencode.jsonc` :

```
{  
  
  "skills": {  
  
    "paths": [".opencode/skills"],  
  
    "urls": []  
  
  }  
  
}
```

OpenCode рекурсивно сканирует указанные директории на наличие `**/SKILL.md` . Силлы из `urls` загружаются при старте.

1.4. Как LLM выбирает скилл

1. В промпте пользователя присутствует ключевое слово, совпадающее с описанием скилла
2. LLM видит список доступных скиллов и их описание (список в системном промпте)
3. LLM вызывает инструмент `skill(name="имя-скилла")`
4. OpenCode загружает тело SKILL.md в контекст сессии

2. Сравнение: скиллы vs другие механизмы правил

2.1. Механизмы правил в проекте AIS

Механизм	Где лежит	Когда загружается	Пример
<code>** .opencode/*.mdc **</code>	<code>.opencode/</code>	Всегда (alwaysApply: true)	<code>preflight.mdc</code> , <code>sql-rules.mdc</code>
<code>AGENTS.md</code>	корень проекта	Всегда (instructions в opencode.jsonc)	Ядро правил проекта
Команды	<code>.opencode/command/*.md</code>	По имени команды <code>/rules</code>	<code>cycle.md</code> , <code>rules.md</code>
Плагины	<code>.opencode/plugins/*.js</code>	Всегда (JS-хуки)	<code>save-prompts.js</code> , <code>loop-guard.js</code>
МСП-серверы	отдельные процессы	По вызову инструмента	<code>ais-catalog</code> , <code>sybase-docs</code>
Скиллы	<code>.opencode/skills/*/SKILL.md</code>	По требованию (description)	Новые

2.2. Ключевое отличие: загрузка по требованию

ВСЕГДА в контексте (alwaysApply)	
• AGENTS.md	
• preflight.mdc ← ≈80 строк	
• russian-language.mdc ← ≈25 строк	
• knowledge-first.mdc ← ≈35 строк	
• save-analysis.mdc ← ≈25 строк	

	• token-economy.mdc	← ≈60 строк	
	• sql-rules.mdc	← ≈120 строк	
	• pb-object-rules.mdc	← ≈150 строк	
	• vss-rules.mdc	← ≈80 строк	
	• jira-table-rules.mdc	← ≈50 строк	
	• ... ещё ~10 mdc-файлов		
	ИТОГО: ≈1000+ строк правил ВСЕГДА в контексте		
	≈ 3000-5000 токенов на каждую сессию		

	ПО ТРЕБОВАНИЮ (skills)		
	• pb-object-skill	← только при РВ-задачах	
	• sql-skill	← только при SQL-задачах	
	• jira-task-skill	← только при задачах Jira	
	• vss-skill	← только при работе с VSS	
	• sborka-skill	← только при СБОРКЕ	
	• gui-wpf-skill	← только при GUI/WPF	
	• taskplan-skill	← только при TaskPlan	
	• security-skill	← только при секретах	
	ИТОГО: ≈2000 строк загружаются ТОЛЬКО при нужде		

| ≈ 0 токенов, когда не нужны |

2.3. Экономия токенов

Сценарий	Без скиллов (всегда в контексте)	Со скиллами (по требованию)
Простой вопрос	3000-5000 токенов правил	1000-1500 токенов (только alwaysApply)
Задача PB	3000-5000 токенов	1000 + 500 (скилл) = 1500
Задача SQL	3000-5000 токенов	1000 + 400 (скилл) = 1400
СБОРКА	3000-5000 токенов	1000 + 600 (скилл) = 1600
Задача VSS + SQL	3000-5000 токенов	1000 + 500 + 400 = 1900

Экономия: 40-70% контекста на правила — это критично при работе с большими задачами.

2.4. Точность: скиллы vs alwaysApply

Критерий	alwaysApply	skills
Доступ к правилам	Всегда	Только при вызове
Риск «забыть» загрузить	Нет (всегда есть)	Да (зависит от description)
Нагрузка на контекст	Всегда максимальная	Минимальная
Гранулярность	Грубая (всё или ничего)	Тонкая (по теме)

Контроль LLM	Пассивный (авто)	Активный (выбор)
--------------	------------------	------------------

Важно: критичные правила (язык, безопасность, кодировка) **не должны** быть скиллами — они всегда Apply.

3. Рекомендации: какие скиллы создать

3.1. Приоритет 1 — критичные скиллы (создать сейчас)

3.1.1. `pb-object-skill` — Правила PowerBuilder объектов

Описание: Загружать при анализе, правке, генерации PB-объектов (.srw, .sru, .srd, .srf, .srm, .srs).

Выигрыш: не нужно грузить pb-object-rules.mdc (≈150 строк) для задач, не связанных с PB.

Содержание:

- Как работать с event-ами (Click, ItemChanged, etc.)
- Правила call super:: и наследования
- HEX-кодирование русского текста (~n, ~t)
- Правила именования: w_, u_, dddw_

- Одна PBL — одна сущность
- Ссылки на `C:\AIS\AI\Prod\.opencode\pb-object-rules.mdc`

Пример SKILL.md:

```
---

name: pb-object-skill

description: Используй ТОЛЬКО при работе с объектами PowerBuilder: анализ .sr* файлов, правка PB-

---

# Правила PowerBuilder объектов

## Базовые правила

- Одна PBL – одна сущность (golden_subject, golden_bill и т.д.)

- Не смешивать объекты из разных PBL

- При правке – проверять файловую кодировку Windows-1251

## Event-ы

[содержимое из pb-object-rules.mdc]

## Функции и переменные

[содержимое из pb-object-rules.mdc]
```

```
## HEX-кодирование
```

```
[содержимое из pb-object-rules.mdc]
```

3.1.2. `sql-skill` — Правила SQL (Sybase ASE)

Описание: Загружать при анализе, правке, генерации SQL-кода для Sybase ASE 15.5.

Выигрыш: sql-rules.mdc (≈120 строк) не будет загружаться для задач без SQL.

Содержание:

- Специфика Sybase ASE 15.5 (min/max, ISNULL, конвертация типов)
- Запрещённые конструкции (iif, ??, ?., =>)
- Правила оформления: отступы, алиасы, WHERE-порядок
- Безопасность данных: только ID, не выводить имена/номера/суммы
- Ссылки на `C:\AIS\AI\Prod\.opencode\sql-rules.mdc`

Пример SKILL.md:

```
name: sql-skill
```

description: Используй ТОЛЬКО при работе с SQL Sybase ASE 15.5: анализ .sql файлов, правка храним

Правила SQL для Sybase ASE 15.5

Специфика ASE

- ISNULL() вместо COALESCE()
- string конкатенация: +
- Дата: getdate(), dateadd(), datediff()

Безопасность данных

[из sql-rules.mdc – раздел безопасности]

Оформление кода

[из sql-rules.mdc]



3.1.3. `jira-task-skill` — Управление задачами Jira

Описание: Загружать при начале/продолжении/завершении задач SYBASE-/SUPRT-, при любых манипуляциях с папками Git_/Test_/Ready_ при ТЗ, PLAN_РЕАЛИЗАЦИИ.

Выигрыш: значительная часть AGENTS.md (раздел «Правило TASK») не нужна при обычных вопросах.

Содержание:

- Правило TASK: корневые пути, именование сессий
- Правило поиска папки (0a): `<задача>_*` потом `<задача>`
- Получение описания из Jira (0b)
- Авто-выгрузка оригиналов в Git_
- Операции с Test_/Ready_/Describe
- ПЛАН_РЕАЛИЗАЦИИ.txt и PlanState.json
- Правило остановки/продолжения
- Стратегия LLM-роутинга
- MCP-знание: `ais-catalog_save_knowledge`
- Ссылки на `C:\AIS\AI\Prod\AGENTS.md` (разделы TASK, Правило остановки, Стратегия LLM)

Пример SKILL.md:

```
---

name: jira-task-skill

description: Используй при ТЕКСТОВОМ упоминании задач SYBASE-*, SUPRT-*, TASK-*, команд TASK/ПРОД

---

# Управление задачами Jira
```

Правило TASK

[ПОЛНЫЙ текст раздела TASK из AGENTS.md]

Правило остановки/продолжения

[текст из AGENTS.md]

Стратегия LLM-роутинга

[текст из AGENTS.md]

3.2. Приоритет 2 — важные скиллы (создать в ближайшее время)

3.2.1. `vss-skill` — Работа с VSS



Описание: Загружать при командах VSS, checkout/checkin, истории, проблемах с VSS.

Содержание:

- Команды VSS (Get, Status, Checkout, Checkin)
- Модуль VSS-Utils.ps1
- Известные ошибки и решения
- Ссылки на `C:\AIS\AI\Prod\.opencode\vss-rules.mdc`

3.2.2. `sborka-skill` — Сборка проекта

Описание: Загружать при командах СБОРКА, при сборке golden.exe + *.pbd.

Содержание:

- Скрипт `Sborka-Golden.ps1`
- Режимы: GALAXY (тестирование), DEBUG
- Версия из `golden.srj`
- Папки выхода: `Exe_yyyy_mm_dd__hh_mm\`
- Процесс: компиляция → PBD → EXE
- Ссылки на `C:\AIS\AI\Prod\docs\services\sborka.md`

3.2.3. `gui-wpf-skill` — WPF-интерфейс (СТАНДАРТ2)

Описание: Загружать при правке GUI-кода, WPF-окон, СТАНДАРТ2, themes.

Содержание:

- Архитектура СТАНДАРТ2 (vs СТАНДАРТ1)

- Runspace + DispatcherTimer
- Metro-темы
- Визуальные стили и control templates
- Ссылки на `C:\AIS\AI\Prod\.opencode\standard2-gui.mdc`

3.2.4. `taskplan-skill` — Управление TaskPlan

Описание: Загружать при командах TaskPl, ТаскПлан, управлении планами.

Содержание:

- Запуск GUI: `Show-TaskPlanGUI.ps1`
- Вкладки: Список задач, Управление, Тесты
- TaskPlan-Tracker.ps1, TaskPlan-Manager.ps1
- Автотесты: `TaskPlan-Tests.ps1`
- OUTPL — анализ ошибок
- Ссылки на `C:\AIS\AI\Prod\docs\taskplan-gui-guide.md`

3.3. Приоритет 3 — специализированные скиллы (по мере необходимости)

3.3.1. `out-obr-skill` — Анализ вывода (ОБР/OUT/OUTPL)

Описание: Загружать при командах ОБР, OUT, OUTPL, при анализе `temp/last_output.log`.

Содержание:

- Формат логов (МОРДА, МОРДА2)
- Распознавание ошибок
- Автоисправление (fix-циклы)
- Ссылки на `C:\AIS\AI\Prod\scripts\Out-LastOutput.ps1`

3.3.2. `security-skill` — Безопасность и секреты

Описание: Загружать при работе с паролями, токенами, `.local_secrets.json`, шифрованием.

Содержание:

- Правила хранения секретов
- `config/.local_secrets.json`
- `Read-SecretFromConfig`
- Запрет на Git-коммит секретов
- Шифрование AES-256
- Ссылки на `C:\AIS\AI\Prod\AGENTS.md` (раздел «Безопасность»)

3.3.3. `sql-diagnostics-skill` — Диагностические SQL-запросы

Описание: Загружать при создании/модификации DIAG-объектов, диагностических MessageBox.

Содержание:

- DIAG-N messagebox в PB
- Только SELECT (ЭКСТРА-режим)
- Безопасность: только ID, не имена/номера/суммы
- Формат DIAG_YYYY_MM_DD_hh_mm

3.3.4. `mcp-skill` — Работа с MCP-серверами

Описание: Загружать при вопросах о настройке MCP, подключении серверов.

Содержание:

- Список MCP-серверов проекта
- `opcode.jsonc` → секция `mcp`

- Установка `uvx.exe mcp-atlassian`
- Переменные окружения
- Ссылки на `C:\AIS\AI\Prod\docs\services\mcp-servers.md`

3.3.5. `plugin-skill` — Плагины OpenCode

Описание: Загружать при разработке, настройке, отладке плагинов (.js).

Содержание:

- Архитектура плагинов (PluginInput → Hooks)
- Хуки: `tool.execute.before`, `config`, `chat.message`
- Существующие плагины проекта
- Ссылки на `C:\AIS\AI\Prod\docs\services\plugins.md`

4. Какие правила НЕ должны стать скиллами

Критичные правила (всегда в контексте):

Файл	Почему alwaysApply
------	--------------------

russian-language.mdc	Язык коммуникации — невозможно «забыть»
russian-mandatory.md	То же
preflight.mdc	Проверки перед записью — критичны всегда
knowledge-first.mdc	Порядок поиска — всегда
save-analysis.mdc	Автосохранение — всегда
save-large-prompts.mdc	Автосохранение больших промптов — всегда
task-save-results.mdc	Фиксация результатов — всегда
token-economy.mdc	Экономия токенов — всегда
index.mdc	Точка входа — всегда
project-context.mdc	Контекст проекта — всегда
repository-layout.mdc	Структура репозитория — всегда
glossary.mdc	Глоссарий — всегда (еньё)

5. Пошаговый план внедрения

5.1. Подготовка

1. **Создать папку** `C:\AIS\AI\Prod\.opencode\skills\` (если не существует)
2. **Добавить в** `C:\AIS\AI\Prod\opencode.jsonc` :

```
{
```

```

"skills": {

    "paths": [".opencode/skills"],

    "urls": []

}

}

```

5.2. Создание скиллов (по приоритету)

Шаг	Скилл	Файл	Статус
1	pb-object-skill	.opencode/skills/pb-object-skill/SKILL.md	✓ Создан
2	sql-skill	.opencode/skills/sql-skill/SKILL.md	✓ Создан
3	jira-task-skill	.opencode/skills/jira-task-skill/SKILL.md	✓ Создан
4	vss-skill	.opencode/skills/vss-skill/SKILL.md	✓ Создан
5	sborka-skill	.opencode/skills/sborka-skill/SKILL.md	✓ Создан
6	gui-wpf-skill	.opencode/skills/gui-wpf-skill/SKILL.md	✓ Создан
7	taskplan-skill	.opencode/skills/taskplan-skill/SKILL.md	✓ Создан
8	out-obr-skill	.opencode/skills/out-obr-skill/SKILL.md	✓ Создан
9	security-skill	.opencode/skills/security-skill/SKILL.md	✓ Создан
10	sql-diagnostics-skill	.opencode/skills/sql-diagnostics-skill/SKILL.md	✓ Создан
11	mcp-skill	.opencode/skills/mcp-skill/SKILL.md	✓ Создан
12	plugin-skill	.opencode/skills/plugin-skill/SKILL.md	✓ Создан

5.3. Миграция

1. Для каждого создаваемого скилла:

- Скопировать содержимое `.mdc`-файла в тело `SKILL.md`
- Написать description с ключевыми словами (front-load)
- Убрать дублирующиеся блоки (ссылки на AGENTS.md вместо копирования)

2. **Убрать из alwaysApply** (убрать `alwaysApply: true` или удалить `.mdc`):

- `pb-object-rules.mdc` → заменить `pb-object-skill`
- `sql-rules.mdc` → заменить `sql-skill`
- `vss-rules.mdc` → заменить `vss-skill`
- `jira-table-rules.mdc` → ВКЛЮЧИТЬ В `jira-task-skill`
- `standard2-gui.mdc` → заменить `gui-wpf-skill`

3. **Оставить alwaysApply** все критичные правила (язык, безопасность, кодировка)

>  **Реализовано 15.09.2026:** папка `.opencode\skills\` создана, блок `skills.paths` добавлен в `opencode.jsonc`, созданы все 12 скиллов. Значения `alwaysApply` в `pb-object-rules.mdc`, `vss-rules.mdc`, `jira-table-rules.mdc`, `standard2-gui.mdc` переведены в `false` (доступ через скиллы). `sql-rules.mdc` оставлен как был (`alwaysApply: false` изначально, критичный раздел «Безопасность данных» продублирован в скилле `sql-skill`). Для применения изменений требуется перезапуск OpenCode.

5.4. Проверка

1. Перезапустить OpenCode
2. Проверить: `skill(name="pb-object-skill")` — должен загрузиться
3. Проверить: контекст сессии уменьшился (в Debugger/логах)

4. Проверить: LLM корректно вызывает скиллы при нужных задачах

6. Дополнительные возможности

6.1. Skills через URLs (удалённые скиллы)

Если в будущем понадобится шарить скиллы между проектами:

```
{
  "skills": {
    "paths": [".opencode/skills"],
    "urls": [
      "https://example.com/opencode-skills/",
      "git@github.com:company/opencode-skills.git"
    ]
  }
}
```

6.2. Скиллы как команда (hybrid)

Можно комбинировать: скилл + команда для явного вызова:

```
# .opencode/command/sborka.md

---

description: "Запуск сборки проекта. Формат: СБОРКА [GALAXY|DEBUG]"

---

Загрузи скилл sborka-skill и выполни сборку с параметрами: $ARGUMENTS
```

6.3. Скиллы и ссылки (references)

Скиллы могут ссылаться на внешние документы через `references` :

```
{
  "references": {
    "ais-docs": {
      "path": "docs",
      "description": "Документация проекта AIS"
    }
  }
}
```

Внутри скилла: см. @ais-docs/taskplan-guide.md

7. Риски и ограничения

Риск	Описание	Решение
LLM не вызовет скилл	Нет гарантии, что описание триггерит	Точное front-loading ключевых слов
Задержка загрузки	Скилл грузится отдельным вызовом	Приемлемо для не-аварийных задач
Дублирование	Частичное дублирование с alwaysApply	Оставить базовые правила alwaysApply, скиллы — детали
Версионирование	Скиллы могут устареть	Хранить в git, версионировать через changelog

8. Заключение

Механизм скиллов — ключ к масштабированию правил проекта без перегрузки контекста. При текущем объеме правил (≈30+ .mdc-файлов, ≈1000 строк alwaysApply) скиллы позволяют:

- **Экономить 40-70% токенов** на правилах в каждой сессии

- **Повышать точность** — LLM получает только релевантные правила
- **Упрощать поддержку** — каждый скилл изолирован в своей папке
- **Шарить знания** — через `skills.urls` между проектами

Приоритет: начать с `pb-object-skill`, `sql-skill`, `jira-task-skill` — они покрывают 80% рабочих сценариев.